PrioGLPI Tecnico

Arquitectura, integracion, seguridad y operacion

Documento de referencia para equipos de tecnologia, operaciones y gobierno de servicios.

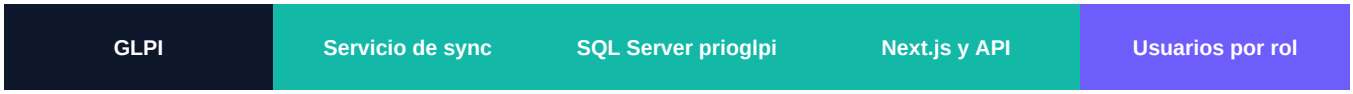
Resumen tecnico

PrioGLPI es una aplicacion web Next.js con TypeScript que integra datos operativos provenientes de GLPI, los sincroniza hacia SQL Server y expone una interfaz y API interna para los perfiles de operacion.

Principio de arquitectura
GLPI conserva el registro del ticket. PrioGLPI agrega una capa operativa para priorizacion, coordinacion, consulta, metricas y auditoria.

Arquitectura

La interfaz y rutas API se ejecutan en una aplicacion Next.js. Un runtime SQL Server soporta repositorios operativos. Procesos programados actualizan informacion desde GLPI; los usuarios trabajan sobre vistas y reglas de acceso propias de PrioGLPI.



Componente	Responsabilidad
Next.js, React y TypeScript	Interfaz web y endpoints internos protegidos.
SQL Server	Cache operativa, ranking, coordinacion, usuarios, auditoria y corridas de sync.
Integracion GLPI	Lectura de tickets, grupos, usuarios, actores y seguimientos configurados.
Workers programados	Actualizacion periodica y registro de cada ejecucion.
Control de release	Validacion de integridad antes de los arranques operativos configurados.

Modelo operativo de datos

La base operacional permite evitar consultas remotas por cada pantalla y conservar el contexto necesario para priorizar y medir.

Entidad	Finalidad
glpi_ticket_cache	Replica operativa de tickets y datos de contexto.
ticket_ranking	Posicion de prioridad y propietario/alcance.
ticket_coordination	Avance, comentario, fecha estimada y usuario actualizador.
app_user / user_sector_scope	Usuarios, roles y alcance sectorial.
sync_run / audit_log	Ejecuciones de sync y evidencia de eventos.
Caches de grupos, usuarios, actores, tareas y followups	Contexto de GLPI para vistas y filtros.

Mapa de API interno

Las rutas se consumen desde la interfaz y se protegen con sesion. No se disena como API publica multi-tenant.

Dominio	Rutas representativas
Sesion	/api/auth/login, /logout, /me
Tickets	/api/tickets, /api/tickets/[id], /followups, /unranked, /meta/ticket-filters
Ranking	/api/rankings, /api/rankings/[ticketId], /api/rankings/reorder
Coordinacion	/api/coordination/[ticketId]
Operacion	/api/sync/run, /api/sync/status, /api/health, /api/activity
Metricas	/api/metrics/coordinator, /referent, /supervisor
Administracion	/api/admin/users, /priorizadores, /audit y export

Seguridad y control de acceso

La aplicacion usa autenticacion local basada en sesion JWT en cookie y aplica autorizacion por rol y alcance. Incluye sanitizacion de secretos en errores y rate limiting de intentos de inicio de sesion. Las credenciales, conexion SQL Server y configuracion GLPI deben permanecer fuera del repositorio en variables de entorno.

• Usar HTTPS y cookies seguras en produccion.

• Asignar usuarios y sectores bajo minima necesidad.

• Auditar cambios de ranking, coordinacion y administracion.

• No exponer secretos, archivos .env ni datos operativos en entregables.

Sincronizacion e integracion

El worker consulta GLPI, normaliza los datos requeridos y persiste caches operativas. sync_run conserva inicio, fin, estado, cantidades y error sanitizado. Las acciones de escritura hacia GLPI se habilitan explicitamente segun el despliegue.

Operacion y despliegue

El arranque web y los sync programados pueden validar el release antes de ejecutar. Un manifiesto de hashes permite detectar alteraciones accidentales del release en el circuito normal de ejecucion. Esto es un control de integridad operativo, no una sustitucion de controles de acceso del sistema operativo.

Observacion de demo
En el entorno relevado, el perfil tecnico puede acceder a vistas de coordinacion y a informacion amplia. Antes de comercializar un modelo de minima visibilidad, validar y ajustar permisos por rol y sector en cada cliente.

Limites y decisiones de adopcion

No se deben prometer capacidades no implementadas: multi-tenant, API publica, alta disponibilidad, recuperacion ante desastre, SSO corporativo o SLA formal requieren diseno y contratacion especifica. La adopcion debe comenzar con configuracion de sectores, responsables, frecuencia de sync, pruebas y criterios de prioridad.